Day 7 — Date & Time Basics

Session	Day 7 of 30
Week	Week 1 — SQL Foundations (FINAL DAY)
Topic	Date & Time: NOW(), INTERVAL, EXTRACT, DATE_TRUNC
Duration	~40 minutes
Level	Beginner → Intermediate
Goal	Write time-based queries for incremental pipeline loads

■ What you will learn today: PostgreSQL date/time types · NOW() and CURRENT_DATE · Date arithmetic with INTERVAL · EXTRACT() and DATE_PART() · DATE_TRUNC() for time bucketing · Casting strings to dates · Computing date differences

Time	Segment	Duration
0:00	Hook — why dates rule every pipeline	3 min
0:03	PostgreSQL date/time types overview	5 min
0:08	NOW(), CURRENT_DATE, CURRENT_TIMESTAMP	4 min
0:12	Date arithmetic with INTERVAL	6 min
0:18	EXTRACT() and DATE_PART()	5 min
0:23	DATE_TRUNC() for bucketing	6 min
0:29	Casting & date differences	4 min
0:33	Hands-on exercise	5 min
0:38	Recap + Week 2 preview	2 min

[0:00] Hook — Why Dates Rule Every Pipeline (3 min)

Camera script:

"Every data pipeline has a timestamp problem. Either you're loading too much — full table every run — or you're loading too little because your date filter is wrong. Today you're going to learn the PostgreSQL date tools that solve this. By the end of this session you will be writing the incremental load watermark query — the single most important date pattern in data engineering."

Talking points:

- Nearly every production table has at least one timestamp column (created_at, updated_at, ingested_at)
- Incremental ETL means: "give me only the new rows since the last run" — this requires date math
- Date bugs are silent — a wrong INTERVAL loads stale data, and no error is thrown
- This is Week 1's final lesson. Everything you've learned this week will combine in the capstone problem.

[0:03] PostgreSQL Date & Time Types (5 min)

Camera script:
"PostgreSQL has four date and time types you'll use regularly. Let's look at each one and when you should reach for it."

Type	Example	Has TZ?	Best For
DATE	2024-06-15	No	Daily partitions, date-only keys
TIMESTAMP	2024-06-15 14:30:00	No	Local timestamps (caution in prod)
TIMESTAMPTZ	2024-06-15 14:30:00+00	Yes	RECOMMENDED for all pipeline tables
INTERVAL	7 days, 3 hours	N/A	Date arithmetic, window calculations

Pipeline rule: Always use TIMESTAMPTZ in production. If your pipeline ever runs in a different region or the server timezone changes, TIMESTAMP (no TZ) will silently return wrong results. TIMESTAMPTZ stores the moment in time unambiguously.

[0:08] Getting the Current Date and Time (4 min)

Camera script:

"PostgreSQL has several built-in functions for the current moment. They look similar, but there are important differences. Let me show them side by side."

```
-- Current date only (no time)
SELECT CURRENT_DATE; -- 2024-06-15

-- Current timestamp WITH timezone (recommended)
SELECT NOW(); -- 2024-06-15 14:30:22.451+00

-- Identical to NOW()
SELECT CURRENT_TIMESTAMP;

-- All four side by side
SELECT
CURRENT_DATE AS just_the_date,
NOW() AS full_timestamp,
CURRENT_TIMESTAMP AS same_as_now,
CURRENT_TIME AS just_the_time;
```

Pipeline pattern — auto-timestamping rows:
CREATE TABLE events (id SERIAL, event_type TEXT, ingested_at TIMESTAMPTZ DEFAULT NOW());
INSERT INTO events (event_type) VALUES ('purchase');
-- ingested_at is set automatically — no application code needed

[0:12] Date Arithmetic with INTERVAL (6 min)

Camera script:
"INTERVAL is how you do math with dates. Add it, subtract it — you get a new timestamp. This is the heart of incremental pipeline queries."

```
-- Basic INTERVAL arithmetic
SELECT
NOW() - INTERVAL '1 hour' AS one_hour_ago,
NOW() - INTERVAL '7 days' AS one_week_ago,
NOW() - INTERVAL '30 days' AS thirty_days_ago,
NOW() + INTERVAL '1 month' AS next_month,
NOW() - INTERVAL '1 day 6 hours' AS compound_interval;
```

The incremental load watermark — the most important date query you will write:

```
-- Production incremental load pattern
-- Run this every hour to get only the last hour's new events
SELECT *
FROM pipeline_events
WHERE ingested_at >= NOW() - INTERVAL '1 hour'
ORDER BY ingested_at;

-- Or parametrize for any window:
WHERE ingested_at BETWEEN NOW() - INTERVAL '1 hour' AND NOW()
```

> **INTERVAL formats PostgreSQL accepts:** '7 days' · '3 hours' · '90 minutes' · '1 day 6 hours' · '2 weeks' · '1 month' · '1 year'

[0:18] EXTRACT() and DATE_PART() (5 min)

Camera script:

"Sometimes you don't need the full timestamp — you just need the hour, or the day of week, or the month. EXTRACT pulls out a single numeric component."

```
SELECT
EXTRACT(YEAR FROM NOW()) AS yr,
EXTRACT(MONTH FROM NOW()) AS mo,
EXTRACT(DAY FROM NOW()) AS dy,
EXTRACT(HOUR FROM NOW()) AS hr,
EXTRACT(MINUTE FROM NOW()) AS mins,
EXTRACT(DOW FROM NOW()) AS day_of_week; -- 0=Sun, 6=Sat

-- DATE_PART() is equivalent, slightly different syntax
SELECT DATE_PART('hour', NOW()) AS hour_num;
```

Pipeline use cases for EXTRACT:

Pattern	Use Case
`EXTRACT(DOW FROM ts) = 0`	Sunday-only pipeline health checks
`EXTRACT(HOUR FROM ts) < 6`	Identify off-hours events (potential anomaly)
`EXTRACT(MONTH FROM ts)`	Monthly partition routing in ETL
`EXTRACT(EPOCH FROM interval)`	Convert interval to seconds for numeric math

[0:23] DATE_TRUNC() — The Bucketing Function (6 min)

Camera script:

"DATE_TRUNC is one of the most powerful functions in time-series data work. It rounds a timestamp DOWN to the start of a given time unit. Think of it as snapping a timestamp to the nearest hour, day, week, or month — always rounding down."

```
SELECT
NOW() AS full_timestamp,
DATE_TRUNC('hour', NOW()) AS start_of_hour,
DATE_TRUNC('day', NOW()) AS start_of_day, -- midnight
DATE_TRUNC('week', NOW()) AS start_of_week, -- Monday
DATE_TRUNC('month', NOW()) AS start_of_month,
DATE_TRUNC('year', NOW()) AS start_of_year;
```

The daily bucketing pattern — used in every metrics pipeline:

```
-- Count events per day (ignores the time portion)
SELECT
DATE_TRUNC('day', event_time) AS event_day,
event_type,
COUNT(*) AS event_count
FROM raw_events
GROUP BY DATE_TRUNC('day', event_time), event_type
ORDER BY event_day DESC, event_count DESC;
```

> **KEY INSIGHT:** The same expression in GROUP BY and SELECT must match exactly. Using DATE_TRUNC in SELECT but event_time::DATE in GROUP BY will cause an error or wrong results. Copy-paste the expression to be safe.

[0:29] Casting Strings & Computing Differences (4 min)

Camera script:

"Two more tools you'll use constantly: casting strings to dates for data cleaning, and computing how much time has elapsed between two timestamps."

```
-- Casting: :: operator (PostgreSQL style)
SELECT '2024-06-15'::DATE AS date_val;
SELECT '2024-06-15 08:30:00'::TIMESTAMP AS ts_val;
SELECT '2024-06-15 08:30:00+00'::TIMESTAMPTZ AS tstz_val;
SELECT NOW()::DATE AS today_no_time;

-- Date subtraction: integer result (number of days)
SELECT '2024-12-31'::DATE - '2024-01-01'::DATE AS days_in_2024;

-- AGE(): human-readable interval
SELECT AGE('2024-12-31'::DATE, '2024-01-01'::DATE); -- 11 mons 30 days

-- EXTRACT(EPOCH): convert interval to seconds, then to any unit
SELECT
ROUND(EXTRACT(EPOCH FROM (NOW() - started_at)) / 60, 1) AS mins_ago,
ROUND(EXTRACT(EPOCH FROM (NOW() - started_at)) / 3600, 1) AS hours_ago
FROM pipeline_runs;
```

Common Error	Cause	Fix
`invalid input syntax for type date`	String format doesn't match ISO 8601	Use YYYY-MM-DD format
`operator does not exist: timestamp - text`	Subtracting a string instead of interval	Add INTERVAL keyword: INTERVAL '7 days'
`column must appear in GROUP BY`	DATE_TRUNC in SELECT but not GROUP BY	Add same expression to GROUP BY

[0:33] Hands-On Exercise (5 min)

> **Task:** Write a query against the pipeline_runs table that returns only runs from the last 30 days. Show run_id, pipeline_name, started_at, and a column hours_ago showing how many hours ago each run started (rounded to 1 decimal). Order by started_at descending.

Solution:

```
SELECT
run_id,
pipeline_name,
started_at,
ROUND(
EXTRACT(EPOCH FROM (NOW() - started_at)) / 3600,
1) AS hours_ago
FROM pipeline_runs
WHERE started_at >= NOW() - INTERVAL '30 days'
ORDER BY started_at DESC;
```

Expected output columns:

```
run_id | pipeline_name | started_at | hours_ago
```

Walk through with students:

- WHERE started_at >= NOW() - INTERVAL '30 days' — the incremental watermark pattern
- EXTRACT(EPOCH FROM ...) — converts a timestamp difference to seconds
- Dividing by 3600 — converts seconds to hours
- ROUND(..., 1) — one decimal place for readability

[0:38] Recap + Week 2 Preview (2 min)

Camera script:

"That's Day 7 — and the end of Week 1. In seven sessions you've gone from zero SQL knowledge to writing production-grade pipeline queries. Let's recap what you can do with dates now, and then I'll show you what's coming in Week 2."

Week 1 complete — you can now:

- ■ SELECT data and alias columns (Day 2)
- ■ Filter rows with WHERE, IS NULL, comparison operators (Day 3)
- ■ Combine conditions with AND, OR, NOT, IN, BETWEEN (Day 4)
- ■ Sort and paginate results with ORDER BY, LIMIT, OFFSET (Day 5)
- ■ Clean and match text with LOWER, TRIM, LIKE, ILIKE (Day 6)
- ■ Work with dates — NOW(), INTERVAL, EXTRACT, DATE_TRUNC (Day 7)

■ Coming next — Week 2: Aggregations & Shaping Data Day 8: COUNT, SUM, AVG, MIN, MAX — aggregate your pipeline data Day 9: GROUP BY — bucket results into categories Day 10: HAVING — filter after aggregation Day 11: CASE expressions — conditional logic and pivot transforms

Camera sign-off:

"If you've made it through Week 1 — congratulations. You now have the foundation every data engineer needs. Week 2 is where things get interesting: we'll start collapsing millions of rows into the summary metrics that power dashboards and pipeline health reports. See you in Day 8."

Appendix — Quick Reference Card: Day 7

Function / Syntax	Returns	Pipeline Use
`CURRENT_DATE`	DATE	Date-only stamps
`NOW()`	TIMESTAMPTZ	ingested_at defaults
`NOW() - INTERVAL '7 days'`	TIMESTAMPTZ	Incremental watermark
`EXTRACT(HOUR FROM ts)`	FLOAT	Hour-of-day grouping
`EXTRACT(DOW FROM ts)`	FLOAT (0–6)	Day-of-week filters
`EXTRACT(EPOCH FROM interval) / 3600`	FLOAT	Elapsed hours (numeric)
`DATE_TRUNC('day', ts)`	TIMESTAMPTZ	Daily bucketing
`DATE_TRUNC('hour', ts)`	TIMESTAMPTZ	Hourly bucketing
`'2024-01-01'::DATE`	DATE	String to date cast
`AGE(ts1, ts2)`	INTERVAL	Human-readable diff
`date1 - date2`	INTEGER (days)	Day count between dates